

Prohibit zero and NULL from being used as null pointer literals

Document #:	P2048R0
Date:	2020-01-12
Project:	Programming Language C++ Evolution Working Group Incubator (EWGI)
Reply-to:	Vittorio Romeo < vittorio.romeo@outlook.com (mailto:vittorio.romeo@outlook.com)>

Contents

- [1 Abstract](#)
- [2 Revision History](#)
- [3 Motivation](#)
 - [3.1 Zero integer literal](#)
 - [3.2 NULL macro](#)
- [4 Example](#)
- [5 Mechanism](#)
- [6 Wording](#)
- [7 Acknowledgments](#)
- [8 References](#)

§ 1 Abstract

This paper proposes removing the `NULL` macro and prohibiting the use of the integer literal `0` as a null pointer literal from C++23, limiting the change to epoch 2023.

§ 2 Revision History

None.

§ 3 Motivation

(This paper assumes that the reader is familiar with the concept of epochs and its goals, which are explained in P1881¹. Sections 3, 7, and 8.3 of the epochs paper provide context required to understand this paper.)

There currently are multiple ways to represent a *null pointer constant* in C++: the integer literal `0`, the standard macro `NULL`, and the keyword `nullptr`. While the latter is preferable in every scenario, due to backward compatibility, both `0` and `NULL` still exist. They are both suboptimal in terms of functionality, and also needlessly increase the complexity and breadth of the language, often resulting in confusion especially to newcomers.

This paper proposes to make `nullptr` the only valid way to represent a *null pointer constant* in C++, leveraging the epochs mechanism discussed in P1881² to avoid breaking backward compatibility.

§ 3.1 Zero integer literal

`0` is a suboptimal way to represent a *null pointer constant* for multiple reasons. Firstly, it is an integer literal, which is most commonly used to represent the numerical value zero rather than an address. Its duality introduces cognitive overhead to source code readers.

Before	After
<pre>module Example; import Something; int main() { something::foo(0); // Unclear whether `foo` // accepts // an integer or address. something::bar(0); // Unclear whether `bar` // accepts // an integer or address. }</pre>	<pre>epoch 2023; module Example; import Something; int main() { something::foo(0); // `foo` clearly accepts an // integer. something::bar(nullptr); // `bar` clearly accepts an // address. }</pre>

Another problem lies in overload resolution. Overloading a function accepting `int*` with one accepting `int` might silently change the meaning of existing code:

Before	After
<pre>void foo(int*); // (0) /* void foo(int); // (1) */ int main() { foo(0); // Calls (0). }</pre>	<pre>void foo(int*); // (0) void foo(int); // (1) int main() { foo(0); // Calls (1). }</pre>

Under epoch 2023, surprising behavior and bugs would be prevented:

Before	After
<pre> module Example; void foo(int*); int main() { foo(0); // OK. } </pre>	<pre> epoch 2023; module Example; void foo(int*); int main() { foo(0); // Compile-time error. } </pre>

3.2 NULL macro

The NULL macro is defined in the C++20 standard as “*an implementation-defined null pointer constant*”³. This gives implementations the freedom to implement NULL as any zero integral literal (e.g. 0, 0L) or as nullptr. Similarly to the overload resolution situation described above, this definition can cause problem when new overloads are added to an existing overload set.

Imagine adding a function accepting either std::nullptr_t, long, int, or int* to an existing overload set that is being invoked with NULL: the behavior is now hard to predict and may vary depending on the standard library implementation being used.

4 Example

Modern code should use nullptr instead of 0 or NULL to maximize readability and prevent suprising overload resolution outcomes between pointer and integral types. epoch 2023 forbids the use of the integer literal 0 in a context where a pointer is required, and the use of NULL altogether:

Before	After
<pre> module Example; void foo(long); // (0) void foo(int*); // (1) int main() { int* p0 = 0; // OK int* p1 = NULL; // OK int* p2 = nullptr; // OK foo(0); // // Ambiguous? foo(NULL); // // Ambiguous? foo(nullptr); // Calls (1) } </pre>	<pre> epoch 2023; module Example; void foo(long); // (0) void foo(int*); // (1) int main() { int* p0 = 0; // Compilation // error int* p1 = NULL; // Compilation // error int* p2 = nullptr; // OK foo(0); // Calls (0) foo(NULL); // Compilation error foo(nullptr); // Calls (1) } </pre>

§5 Mechanism

In layman’s terms, this is what the compiler would do in a module unit targeting epoch 2023:

- Any use of the standard `NULL` macro would be ill-formed. Ideally, the error would explain that `NULL` is a relic of the past and that `nullptr` should be used instead.
- Any use of the integer literal `0` (or similar, such as `0L`) in a context where a pointer type is required would be ill-formed.
 - During overload resolution, `0` will find overloads accepting integral types as suitable, but will never select any overload accepting a pointer type.

§6 Wording

Will be provided if the paper is positively received. Minor changes are required in `[support.types.nullptr]` and `[conv.ptr]`.

§7 Acknowledgments

None.

§8 References

1. <https://wg21.link/P1881> (<https://wg21.link/P1881>)↵
2. <https://wg21.link/P1881> (<https://wg21.link/P1881>)↵
3. <http://eel.is/c++draft/support.types.nullptr> (<http://eel.is/c++draft/support.types.nullptr>)↵